

Role-Based Access Control (RBAC)

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere i quattro oggetti fondamentali del RBAC: `Role`, `ClusterRole`, `RoleBinding`, `ClusterRoleBinding`
- Distinguere gli oggetti namespace-scoped da quelli cluster-scoped e quando usare ciascuno
- Creare Role e ClusterRole custom applicando il principio del minimo privilegio
- Assegnare ruoli a utenti, gruppi e ServiceAccount tramite `oc adm policy` e manifest YAML
- Verificare i permessi effettivi con `oc auth can-i` e `oc policy who-can`
- Integrare i gruppi Azure AD con i RoleBinding OpenShift su ARO

Teoria e Concetti Chiave

I Quattro Oggetti RBAC

Oggetto	Scope	Funzione
<code>Role</code>	Namespace	Definisce un insieme di permessi validi solo nel namespace dove è creato
<code>ClusterRole</code>	Cluster	Definisce permessi validi in tutto il cluster (o in qualsiasi namespace se referenziato da RoleBinding)
<code>RoleBinding</code>	Namespace	Collega un <code>Role</code> o <code>ClusterRole</code> a soggetti (User, Group, ServiceAccount) nel namespace
<code>ClusterRoleBinding</code>	Cluster	Collega un <code>ClusterRole</code> a soggetti in tutto il cluster

💡 **Suggerimento:** Un `ClusterRole` può essere referenziato da un `RoleBinding` namespace-scoped per riusare una definizione di permessi comune in più namespace, senza dare accesso globale. Questa è una best practice comune per i ruoli standard come `view` o `edit`.

Anatomia di una Rule RBAC

Le regole (**rules**) di un **Role** o **ClusterRole** specificano tre elementi:

Campo	Valori tipici	Descrizione
apiGroups	"" (core), "apps", "batch", "route.openshift.io"	API group delle risorse (vuoto = core API group)
resources	pods, deployments, services, secrets, configmaps	Tipo di risorsa Kubernetes/OpenShift
verbs	get, list, watch, create, update, patch, delete, deletecollection	Operazioni consentite

```
rules:
- apiGroups: ["" ]                # ← "" = core API (Pod, Service, ConfigMap,
  Secret...)
  resources: ["pods", "pods/log"]  # ← risorse e sotto-risorse
  verbs: ["get", "list", "watch"]  # ← sola lettura
- apiGroups: ["apps"]            # ← API group apps (Deployment, ReplicaSet,
  DaemonSet...)
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "update", "patch"]
```

Ruoli Predefiniti in OpenShift

OpenShift include **ClusterRole** predefiniti per i casi d'uso più comuni:

ClusterRole	Descrizione	Uso consigliato
cluster-admin	Accesso completo a tutto il cluster	Solo per amministratori di sistema
admin	Accesso completo al namespace (incluso RBAC del namespace)	Responsabili di progetto
edit	Crea/modifica/elimina la maggior parte delle risorse del namespace	Sviluppatori
view	Sola lettura su tutte le risorse del namespace	QA, monitoring, auditor
self-provisioner	Crea nuovi Project (namespace)	Assegnato di default a tutti gli utenti autenticati
basic-user	Informazioni di base sul cluster (solo se non privato)	Utenti base

⚠ Attenzione: Il ruolo **self-provisioner** è assegnato di default al gruppo **system:authenticated:oauth**. In ambienti enterprise, questo ruolo andrebbe rimosso e la creazione dei namespace gestita da un processo approvato:

```
oc adm policy remove-cluster-role-from-group self-provisioner
system:authenticated:oauth
```

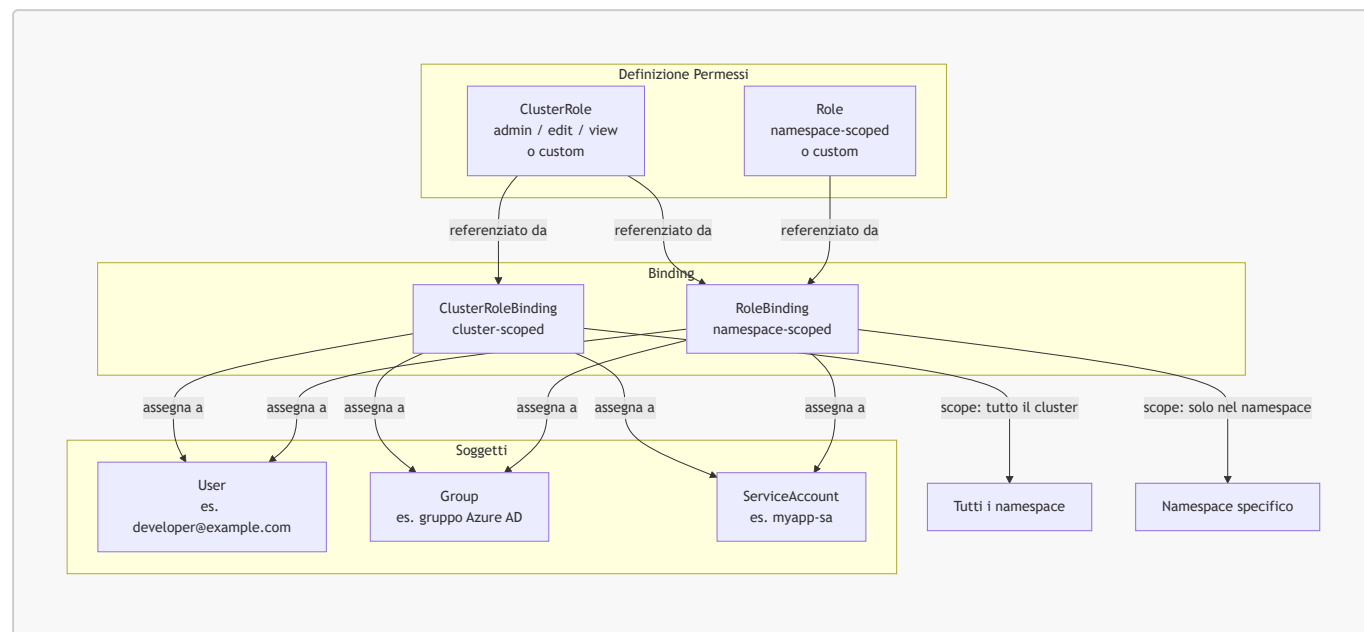
Aggregated ClusterRoles

OpenShift (e Kubernetes 1.9+) supporta ClusterRole **aggregate**: un ClusterRole "padre" aggrega automaticamente le regole di tutti i ClusterRole che hanno una determinata label. Questo permette di estendere i ruoli predefiniti senza modificarli direttamente:

```
# Questo ClusterRole aggiunge automaticamente le sue rules al ClusterRole "view"
# grazie alla label aggregation-rule
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: custom-resource-viewer
  labels:
    rbac.authorization.k8s.io/aggregate-to-view: "true" # ← aggiunge regole a
    "view"
rules:
- apiGroups: ["example.com"]
  resources: ["myresources"]
  verbs: ["get", "list", "watch"]
```

Diagrammi

Gerarchia RBAC: Role/ClusterRole → Binding → Soggetti



Comandi Pratici con Output Atteso

```
# Elencare tutti i ClusterRole disponibili nel cluster
oc get clusterroles | grep -v "^system:"

# Output atteso (selezione):
# NAME                                     CREATED AT
# admin                                   2024-01-15T10:00:00Z
# cluster-admin                           2024-01-15T10:00:00Z
# edit                                    2024-01-15T10:00:00Z
# self-provisioner                         2024-01-15T10:00:00Z
# view                                    2024-01-15T10:00:00Z
```

```
# Assegnare il ruolo "edit" all'utente developer nel namespace myproject
oc adm policy add-role-to-user edit developer -n myproject

# Output atteso:
# clusterrole.rbac.authorization.k8s.io/edit added: "developer"
```

```
# Assegnare il ruolo "view" a un gruppo Azure AD nel namespace myproject
oc adm policy add-role-to-group view \
  "aaaabbbb-cccc-dddd-eeee-ffffgggghhhh" \
  -n myproject

# Output atteso:
# clusterrole.rbac.authorization.k8s.io/view added: "aaaabbbb-cccc-dddd-eeee-ffffgggghhhh"
```

```
# Assegnare il ruolo cluster-admin a un utente (attenzione: privilegio massimo)
oc adm policy add-cluster-role-to-user cluster-admin admin

# Output atteso:
# clusterrole.rbac.authorization.k8s.io/cluster-admin added: "admin"
```

```
# Verificare se l'utente corrente può creare Pod nel namespace myproject
oc auth can-i create pods -n myproject
```

```
# Output atteso (se ha il permesso):
# yes
```

```
# Output atteso (se non ha il permesso):
# no
```

```
# Verificare i permessi di un utente specifico (richiede cluster-admin)
oc auth can-i create deployments --as developer -n myproject
```

```
# Output atteso:
# yes
```

```
# Elencare chi può eliminare Pod nel namespace myproject
oc policy who-can delete pods -n myproject
```

```
# Output atteso:
# Users:  admin, system:admin
# Groups: system:cluster-admins, system:masters
```

```
# Visualizzare tutti i RoleBinding nel namespace myproject
oc get rolebindings -n myproject -o wide
```

```
# Output atteso:
```

NAME	ROLE	AGE	USERS	GROUPS
SERVICEACCOUNTS				
admin	ClusterRole/admin	5d	admin	
developer-edit	ClusterRole/edit	2d	developer	
azure-group-view	ClusterRole/view	1d		aaaabbbb-cccc-dddd-eeee-ffffgggghhhh

```
# Descrivere un RoleBinding per vedere tutti i dettagli
oc describe rolebinding developer-edit -n myproject
```

```
# Output atteso:
# Name:      developer-edit
# Namespace:  myproject
# Labels:     <none>
# Annotations: <none>
# Role:
#   Kind: ClusterRole
```

```
# Name: edit
# Subjects:
# Kind      Name      Namespace
# ----      ----      -
# User      developer
```

Esempi YAML Completi

Role Custom + RoleBinding namespace-scoped

```
# Role custom per sviluppatori: può gestire Deployment ma non Secret
# Scope: solo nel namespace "myproject"
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployment-manager
  namespace: myproject          # ← questo Role esiste solo in myproject
  labels:
    app: myapp
    team: backend
rules:
# Permessi su Deployment e ReplicaSet (API group "apps")
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
# Permessi di sola lettura su Pod (API group "" = core)
- apiGroups: [""]
  resources: ["pods", "pods/log", "pods/status"]
  verbs: ["get", "list", "watch"]
# Permessi su ConfigMap ma NON su Secret
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
# Permessi di eseguire oc rollout restart
- apiGroups: ["apps"]
  resources: ["deployments/rollout"]
  verbs: ["update"]
---
# RoleBinding: assegna il Role "deployment-manager" all'utente "developer"
# nel namespace "myproject"
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: developer-deployment-manager
  namespace: myproject          # ← stesso namespace del Role
subjects:
# Soggetto 1: utente singolo
- kind: User
  name: developer@example.com    # ← nome utente OpenShift (case-sensitive)
  apiGroup: rbac.authorization.k8s.io
# Soggetto 2: gruppo Azure AD
- kind: Group
  name: "team-backend-developers" # ← nome del gruppo (o GUID del gruppo Azure AD)
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role                    # ← "Role" per namespace-scoped, "ClusterRole"
```



```

per cluster-wide
  name: deployment-manager          # ← nome del Role definito sopra
  apiGroup: rbac.authorization.k8s.io

```

ClusterRole Custom + ClusterRoleBinding

```

# ClusterRole custom: accesso in sola lettura ai nodi e alle metriche del cluster
# Usato tipicamente per strumenti di monitoring
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
  labels:
    managed-by: platform-team
rules:
# Sola lettura sui nodi del cluster
- apiGroups: [""]
  resources: ["nodes", "nodes/metrics", "nodes/stats", "nodes/proxy"]
  verbs: ["get", "list", "watch"]
# Sola lettura sui namespace (per listing globale)
- apiGroups: [""]
  resources: ["namespaces"]
  verbs: ["get", "list", "watch"]
# Metriche (API metrics.k8s.io)
- apiGroups: ["metrics.k8s.io"]
  resources: ["nodes", "pods"]
  verbs: ["get", "list"]
---
# ClusterRoleBinding: assegna node-reader al ServiceAccount monitoring-agent
# in tutti i namespace del cluster
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: monitoring-node-reader
subjects:
- kind: ServiceAccount
  name: monitoring-agent          # ← nome del ServiceAccount
  namespace: monitoring          # ← namespace del ServiceAccount
roleRef:
  kind: ClusterRole
  name: node-reader              # ← ClusterRole definito sopra
  apiGroup: rbac.authorization.k8s.io

```

Note Specifiche per Azure ARO

Binding di Gruppi Azure AD a ClusterRole su ARO

Su ARO, dopo aver configurato Azure AD come OIDC provider, i gruppi Azure AD sono disponibili come soggetti dei RoleBinding. Il campo `name` del soggetto deve corrispondere al claim `groups` nel token JWT (solitamente il GUID del gruppo Azure AD):

```
# Trovare il GUID del gruppo Azure AD
az ad group show --group "OpenShift-Admins" --query id -o tsv

# Output atteso:
# aaaabbbb-cccc-dddd-eeee-ffffgggghhhh

# Assegnare cluster-admin al gruppo Azure AD
oc adm policy add-cluster-role-to-group cluster-admin \
  "aaaabbbb-cccc-dddd-eeee-ffffgggghhhh"

# Output atteso:
# clusterrole.rbac.authorization.k8s.io/cluster-admin added: "aaaabbbb-cccc-dddd-eeee-ffffgggghhhh"
```

 **Suggerimento:** In ARO, Azure Red Hat usa la service principal del cluster per alcune operazioni interne. Non modificare i `ClusterRoleBinding` dei soggetti `system:serviceaccount:openshift-*` e `system:masters`: sono critici per il funzionamento del cluster gestito.

Rimozione del Self-Provisioner su ARO

In ambienti enterprise su ARO, la creazione di Project (namespace) deve essere controllata:

```
# Rimuovere il ruolo self-provisioner dal gruppo di tutti gli utenti autenticati
oc adm policy remove-cluster-role-from-group \
  self-provisioner \
  system:authenticated:oauth

# Verificare la rimozione
oc describe clusterrolebinding self-provisioners

# Creare un processo approvato: solo gli admin possono creare namespace
# (ad es. tramite Azure DevOps pipeline o GitOps)
```

Audit RBAC su ARO

```
# Verificare tutti i ClusterRoleBinding che danno cluster-admin
oc get clusterrolebindings \
```

```
-o json | jq '.items[] | select(.roleRef.name=="cluster-admin") |  
{binding: .metadata.name, subjects: .subjects}'
```

Riepilogo dei Punti Chiave

- Il RBAC si basa su quattro oggetti: **Role/ClusterRole** (definiscono permessi) e **RoleBinding/ClusterRoleBinding** (assegnano permessi a soggetti)
 - **Role** e **RoleBinding** sono **namespace-scoped**; **ClusterRole** e **ClusterRoleBinding** sono **cluster-scoped**
 - Un **ClusterRole** può essere riusato in più namespace tramite **RoleBinding** senza dare accesso globale: pattern consigliato per i ruoli standard
 - OpenShift include ruoli predefiniti: **cluster-admin**, **admin**, **edit**, **view**, **self-provisioner** — usarli prima di creare custom role
 - Verificare i permessi con **oc auth can-i** (per l'utente corrente) e **oc policy who-can** (per scoprire chi ha un permesso)
 - Su ARO, i **gruppi Azure AD** (referenziati tramite GUID) possono essere soggetti diretti dei RoleBinding dopo la configurazione OIDC
 - Il ruolo **self-provisioner** assegnato di default a tutti gli utenti autenticati va rimosso in ambienti enterprise per controllare la creazione dei namespace
-

Domande di Verifica

1. Un utente ha bisogno di creare e modificare Deployment nel namespace **dev**, ma non deve avere accesso ad altri namespace. Quale combinazione è corretta?

- A) **ClusterRole** + **ClusterRoleBinding**
- B) **Role** nel namespace **dev** + **RoleBinding** nel namespace **dev**
- C) **ClusterRole** + **RoleBinding** nel namespace **dev** ✓
- D) **Role** nel namespace **dev** + **ClusterRoleBinding**

2. Quale comando verifica se il ServiceAccount **myapp-sa** nel namespace **myproject** può creare Secret nel namespace **myproject**?

- A) `oc policy who-can create secrets -n myproject`
- B) `oc auth can-i create secrets --as myapp-sa -n myproject`
- C) `oc auth can-i create secrets --as system:serviceaccount:myproject:myapp-sa -n myproject` ✓
- D) `oc get rolebindings -n myproject | grep myapp-sa`

3. Cosa fa `oc adm policy add-role-to-user edit developer -n myproject`?

- A) Crea un **Role** chiamato **edit** nel namespace **myproject** per l'utente **developer**
- B) Crea un **RoleBinding** che assegna il **ClusterRole** **edit** all'utente **developer** nel namespace **myproject** ✓
- C) Crea un **ClusterRoleBinding** che assegna il **ClusterRole** **edit** all'utente **developer**
- D) Modifica il **ClusterRole** **edit** aggiungendo l'utente **developer** come soggetto

4. In OpenShift, il **ClusterRole** **self-provisioner** è assegnato di default a quale gruppo?

- A) **system:cluster-admins**
- B) **system:masters**
- C) **system:authenticated:oauth** ✓
- D) **system:serviceaccounts**

5. Su ARO con Azure AD integrato, quale valore deve essere usato come **name** del soggetto in un **RoleBinding** per riferirsi a un gruppo Azure AD?

- A) Il nome display del gruppo (es. **OpenShift-Admins**)
- B) Il GUID del gruppo Azure AD (es. **aaaabbbb-cccc-dddd-eeee-ffffgggghhhh**) ✓
- C) Il UPN dei membri del gruppo (es. **user@company.com**)
- D) Il SID del gruppo Active Directory